MODULE 06

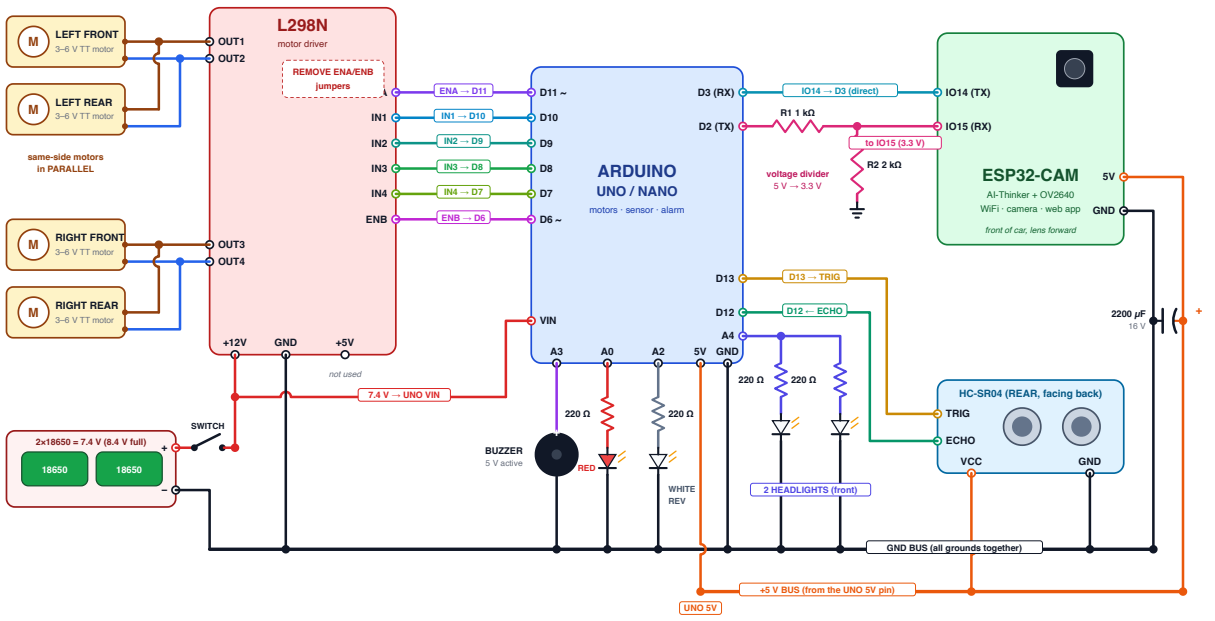
06

Technical Reference

Protocol, safety rules, steering and alarm maths, code structure and extension ideas.

ROBOT CAR – SCHEMATIC WIRING DIAGRAM

4WD chassis · L298N · Arduino UNO/Nano · ESP32-CAM · HC-SR04 (rear) · buzzer + rear LEDs · 2 headlights · 2×18650 (no buck converter: the UNO makes the 5 V)



POWER — no buck converter needed

- Battery 7.4 V → switch → L298N +12V. A 2nd wire in the same +12V terminal → UNO VIN.
- The UNO regulator makes the 5 V: UNO 5V pin → 5 V bus (ESP32-CAM, HC-SR04, buzzer, LEDs).
- 2200 µF 16 V capacitor right at the ESP32-CAM 5V/GND pins (+ leg to 5V).
- L298N: keep the 5V-EN jumper ON, remove ENA/ENB jumpers, leave its +5V pin unused.
- Do not add a servo to this 5 V bus (the UNO regulator is too small for it).

WIRE COLOURS

Battery +7.4 V · +5 V · GND · Motor wires · UART ESP→UNO · UART UNO→ESP · TRIG · ECHO · Buzzer · Headlights

- Wires crossing with a small hop are NOT connected. A black dot = connected.
- **ALL grounds must be common. Never feed 7.4 V into the ESP32-CAM or UNO 5V pin.**

Name _____ Class / Group _____ Date _____

MODULE 06

1 How the system works

This reference explains what happens inside the code: the messages between the phone, the ESP32-CAM and the Arduino, the safety rules, and the maths behind driving and the parking alarm. Use it for oral exams, reports and your own improvements.

LEARNING OBJECTIVES · ABOUT 45 MIN

By the end of this module you can:

- Describe the data flow between phone, ESP32-CAM and Arduino
- Explain a simple text protocol with a checksum
- Explain differential (tank) steering and the parking-alarm timing
- Plan an extension of the project

Data flow

```
Phone browser ──WebSocket ws://192.168.4.1/ws (text)──► ESP32-CAM ──UART 38400──► Arduino UNO
Phone browser ◄──────────────────────────────────────── ESP32-CAM ◄──────────────── Arduino UNO
PC  /view     ◄── same WebSocket (data) ──────────────── ESP32-CAM
Phone + PC <img> ◄── MJPEG http://192.168.4.1:81/stream — ESP32-CAM  (max 3 screens)
```

Messages

Phone → ESP32-CAM (WebSocket)

Message	Meaning	Handled by
`M,<left>,<right>`	Motor speeds −255…255, sent every 100 ms	forwarded to UNO
`H,0|1`	Horn off/on	forwarded
`C,warn,stop,brake,always,mute`	Alarm settings (cm, cm, 0/1, 0/1, 0/1)	forwarded (repeated every 3 s)
`L,0|1|2`	Headlights off / on / hazard	forwarded (repeated every 3 s)
`K,0–255`	Cam light: brightness of the ESP32-CAM flash LED	ESP32
`F,n` / `Q,n`	Camera frame size / JPEG quality	ESP32
`V,0|1` / `X,0|1`	Flip / mirror image	ESP32
`R,phone` / `R,view`	The page says what it is: a driving phone or a PC viewer (for the counters)	ESP32

ESP32-CAM → UNO (serial, with checksum)

The same `M`, `H`, `C`, `L` and `S` lines, plus `*XX` = the XOR of every character before the `*`, in hex. Example: `M,0,0*4D`. The UNO's software serial port can lose bytes while it is transmitting. A damaged line fails the checksum and is ignored, and the next one arrives 150 ms later.

UNO → ESP32-CAM → phone

Message	Meaning
`D,<angle>,<cm>,<flags>`	Every 60 ms. angle = 90 (fixed sensor) or 30–150 (servo). cm = 0 means nothing in range. flags (add them up): 1 reversing · 2 alarm · 4 auto-brake · 8 horn · 16 headlights lit · 32 hazard
`T,l,r,light,horn,flash`	ESP32 → every screen, every 250 ms: motor speeds, headlight mode, horn and cam light, so all screens stay in sync
`I,rssi,phones,uno_ok,viewers`	ESP32 status every 2 s: WiFi signal, number of phones, UNO answering, number of PC viewers

Web addresses

Address	Returns
`http://192.168.4.1/`	the phone app
`/view`	the PC classroom viewer
`/status`	JSON with distance, flags, lights, camera settings
`/cmd?c=M,150,150`	send a command over plain HTTP (for other apps, e.g. MIT App Inventor)
`/capture`	one JPEG photo
`:81/stream`	live MJPEG video

MODULE 06

2 Inside the code

Safety rules (fail-safes)

Where	Rule
Phone	Sends stop when the joystick is released or the page is hidden
ESP32-CAM	No command from the phone for 500 ms → sends stop
Arduino	No command for 600 ms → motors off, horn off
Arduino	Reversing and distance ≤ stop distance and auto-brake on → backwards speed forced to 0
Arduino	Speed ramps 18 PWM steps every 10 ms → no current spikes that could reset the ESP32

Differential (tank) steering

The joystick gives **y** (forward/back) and **x** (left/right) between −1 and 1. Each side's speed is:

```
left  = y + x · turn        right = y − x · turn
divide both by max(1, |left|, |right|)   // keep within −1…1
PWM   = value × 255 × speed%
```

Push straight up and both sides are equal, so the car goes straight. Push right and the left side is faster, so the car turns right. Push fully right with y = 0 and the sides are opposite, so the car spins on the spot.

Parking alarm timing

```
if distance ≤ stop + 5 cm  →  continuous tone
else period = map(distance, stop+5 … warn, 120 ms … 900 ms)
     buzzer ON for the first half of each period
```

Closer means a shorter period, so the beeping is faster, exactly like a real car. The distance is the **median of the last 3 readings**, which removes single wrong echoes.

Files

File	Main parts
`firmware/uno_controller/uno_controller.ino`	`setSide()` motor driver · `rampTo()` soft start · `pingCm()`, `median3()` sensor · `updateAlarm()` · `handleLine()`, `checksumOk()` protocol
`firmware/esp32cam_robot/esp32cam_robot.ino`	`cameraInit()` · `streamHandler()` video · `wsHandler()`, `wsBroadcast()` WebSocket · `readUno()`, `sendToUno()`, `linkSend()`
`firmware/esp32cam_robot/config.h`	WiFi name and password, pins, camera options
`web/index.html`	Phone app: joystick, video, radar canvas, headlights, settings, demo mode

File	Main parts
`web/view.html`	PC viewer: video, photo/video recording, radar, car status, chart, events
`tests/01…06`	Small test sketches for each part (05 = camera only, 06 = bench parking alarm)

Ideas for your own improvements

Scanning radar
Mount the HC-SR04 on an SG90 servo (A1) and set `USE_SERVO 1`. The phone radar then shows a real sweep with blips.

Front collision stop
Add a second HC-SR04 at the front (free pins: D4, D5) and stop forward motion near obstacles.

Battery gauge
Measure the battery with a 10k/4.7k divider on A5 and show the voltage in the app with a new message `B,…`.

Your own app
Build an MIT App Inventor app that sends `/cmd?c=M,…` requests and shows `/status`.